

STACK OPERATIONS AND AMORTIZED COST ANALYSIS

Aim

To implement stack operations (Push, Pop and Top) and analyze their amortized cost using a dynamically resizing array.

Algorithm

1. Initialize the stack with an array of size 2 and set top as -1.
2. Read the number of operations from the user.
3. For a Push operation, resize the array if it is full and insert the element.
4. For a Pop operation, remove the top element if the stack is not empty.
5. For a Top operation, display the top element if the stack is not empty.
6. Display the cost of each operation and maintain the total amortized cost.

Program

```
import java.util.Scanner;

class StackAmortizedFriend {
    static int[] stack = new int[2], top = {-1};
    static int totalCost = 0, pushCount = 0, popCount = 0, topCount = 0;

    static int push(int value) {
        int cost = 1;
        if (top[0] == stack.length - 1) {
            int[] temp = new int[stack.length * 2];
            for (int i = 0; i <= top[0]; i++) { temp[i] = stack[i]; cost++; }
            stack = temp;
        }
        stack[++top[0]] = value;
        totalCost += cost; pushCount++;
        return cost;
    }

    static int pop() {
        if (top[0] == -1) { System.out.println("Stack is empty"); return 0; }
        top[0]--; popCount++; totalCost++; return 1;
    }

    static int topElement() {
        if (top[0] == -1) { System.out.println("Stack is empty"); return 0; }
        System.out.println("Top element = " + stack[top[0]]); topCount++; totalCost++; return 1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of operations: ");
        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            System.out.println("
Operation " + i);
            System.out.print("Enter choice: ");
            int c = sc.nextInt();
            int cost = 0;
            switch (c) {
                case 1: cost = push(sc.nextInt()); System.out.print("1. Push ");
                case 2: cost = pop(); if (cost > 0) System.out.print("2. Pop ");
                case 3: cost = topElement(); if (cost > 0) System.out.print("3. Top ");
                default: System.out.println("Invalid choice");
            }
            System.out.println("Cost of operation " + i + " is " + cost);
        }
        System.out.println("Total amortized cost = " + totalCost);
    }
}
```

```
        sc.close();  
    }  
}
```

Output

Enter number of operations: 3

Operation 1

1. Push

2. Pop

3. Top

Enter choice: 1

Enter value: 10

Cost of PUSH = 1

Operation 2

1. Push

2. Pop

3. Top

Enter choice: 2

Cost of POP = 1

Operation 3

1. Push

2. Pop

3. Top

Enter choice: 3

Stack is empty

Result

Thus, the program to implement stack operations and analyze amortized cost using a dynamically resizing array has been executed successfully. The Push, Pop and Top operations were performed correctly and their costs were determined.